

Linux & Git Cheat Sheet

Online Activity #2
My own quick reference for the Linux terminal and Git

Name: **Eric Nathan A. Delacruz**

Student number: **202304567**

Course, year & section: **BSIT 4-5**

Date: **September 16, 2026**

1. Basic Linux commands

Command	What it does	Sample usage
pwd	Shows the folder I'm in (full path)	\$ pwd
ls	Lists files (-l details, -a hidden too)	\$ ls -la
cd	Goes into a folder (cd .. goes up one)	\$ cd ~/projects
mkdir	Makes a folder (-p makes parents too)	\$ mkdir -p site/css
touch	Creates a new, empty file	\$ touch index.html
cp	Copies files (-r for whole folders)	\$ cp -r site backup
mv	Moves or renames a file or folder	\$ mv old.txt new.txt

Stuck? Type `man <command>` or `<command> --help` to see every option.

Command	What it does	Sample usage
rm	Deletes files (-r folders). No recycle bin!	\$ rm -r temp
cat	Prints everything inside a file	\$ cat notes.txt
echo	Prints text; > saves it into a file	\$ echo "Hi" > hi.txt
grep	Finds text in files (-r folders, -n line no.)	\$ grep -rn "TODO" .
chmod	Changes permissions (+x = can run)	\$ chmod +x run.sh
sudo	Runs a command as admin (root)	\$ sudo apt update
man	Opens a command's manual (q quits)	\$ man ls

2. Basic Git commands

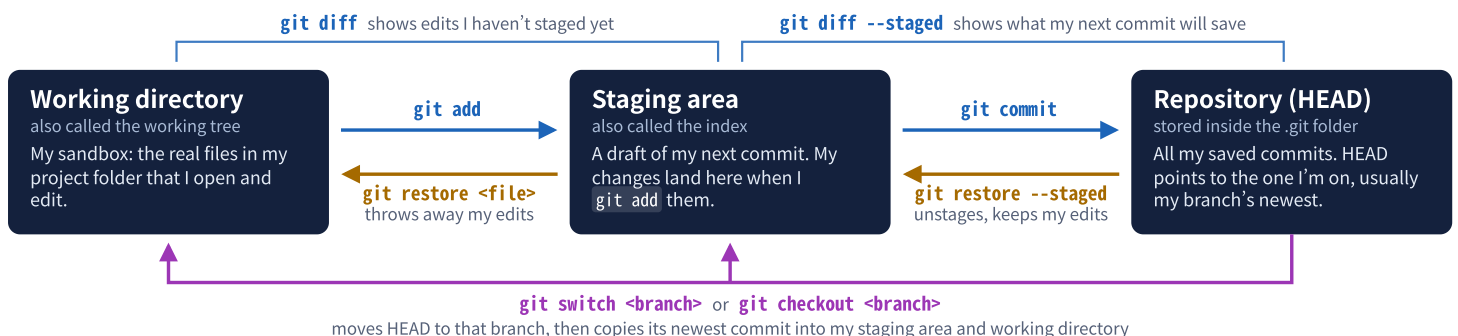
Each color lane groups commands by job, like the branch lines in `git log --graph`.

	Command	What it does (in my words)	Sample usage	Common options / flags
Get a repo	git init	Turns this folder into a repo (adds a hidden <code>.git</code> folder that stores the history).	\$ git init -b main	-b <name> name the first branch (for servers) -q quiet
	git clone	Downloads a full copy of an online repo (files + history) and saves its link as <code>origin</code> .	\$ git clone <repo-url>	-b <branch> start on that branch --depth 1 latest commit only --single-branch one branch only
Record changes	git status	Tells me my current branch and which files are untracked, modified, or staged.	\$ git status -s	-s short view (?? untracked, M modified) in -s --ignored show ignored files
	git add	Puts my changes in the staging area so they get saved in the next commit.	\$ git add index.html \$ git add .	. everything here -A all changes, incl. deletions -p pick parts of a file -u tracked files only
	git diff	Shows the exact lines I added (+) or removed (-) that aren't staged yet.	\$ git diff \$ git diff --staged	--staged (or --cached) staged vs. last commit summary only main dev compare 2 branches
	git commit	Saves my staged changes as a new snapshot, with a short message.	\$ git commit -m "Add login"	-m "msg" message inline -a stage changed tracked files first --amend redo the last commit
History & undo	git log	Lists past commits, newest first, with each one's ID, author, date, and message.	\$ git log --oneline --graph	--oneline one line each --graph branch lines --all all branches -n 5 last 5 --author="Eric" mine
	git restore	Discards my edits or unstages a file. Discarded edits are gone for good!	\$ git restore style.css \$ git restore --staged app.js	--staged unstage, keep edits --worktree discard edits (default) --source=<commit> use an older version
	git tag	Labels a commit with a name like v1.0. Upload tags with <code>git push --tags</code> .	\$ git tag -a v1.0 -m "Final"	-a annotated (with tagger + message) -m message -l "v1.*" list matching -d delete
Branching	git branch	Lists, creates, renames, or deletes branches. Making one doesn't move me onto it.	\$ git branch login-page	-a include remote ones -m <new> rename current -d delete (merged) -D force delete -v show latest commit
	git checkout git switch	Moves me to another branch and updates my files. <code>switch</code> is the newer, branch-only command; <code>checkout</code> can also restore files.	\$ git switch -c login-page \$ git checkout -b login-page	-c <name> (switch) or -b <name> (checkout) create a branch and go to it - back to the previous branch --detach <commit> view an old commit
	git merge	Brings the commits from another branch into the branch I'm on now.	\$ git switch main \$ git merge login-page	--no-ff always make a merge commit --squash merge as one change --abort cancel a conflicted merge

First time on a new computer? Tell Git who I am before committing: `git config --global user.name "Eric Nathan A. Delacruz"` (then the same with `user.email`)

3. The three-tree architecture of Git

Git keeps my project in three "trees" (sets of files). Commands move changes between them, and `git status` shows me where they don't match.



References

Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. <https://git-scm.com/book/en/v2>
Chacon, S., & Straub, B. (2014). Git tools: Reset demystified. In *Pro Git* (2nd ed.). Apress. <https://git-scm.com/book/en/v2/Git-Tools-Reset-Demystified>
Git. (n.d.). *Git reference documentation*. Retrieved September 16, 2026, from <https://git-scm.com/docs>
Free Software Foundation. (n.d.). *GNU Coreutils manual*. <https://www.gnu.org/software/coreutils/manual/>
Kerrisk, M. (n.d.). *Linux man pages online*. man7.org. <https://man7.org/linux/man-pages/>
Built-in help: `git <command> --help`, `man <command>`, and `<command> --help`

AI use declaration

The author used Claude Opus 5 (Anthropic) to draft the command descriptions and sample usages, explain the three-tree architecture, and design the layout of this cheat sheet during assignment preparation. The author reviewed and edited the content as needed and takes full responsibility for the final content of the work.